# Constraining LLM-Assisted Behavior-Driven Development with Script-Enforced Specification Contracts: Design and Evaluation of SpecEngine

Yilun Tsai
*In-Service Master's Program in Artificial Intelligence and Information Systems*
*Department of Computer Science and Information Engineering*
*National Cheng Kung University*
Tainan, Taiwan
p77151150@gs.ncku.edu.tw

***Abstract*—Large language model (LLM) coding agents fill gaps in under-specified requirements with unstated assumptions, and when tests and specification conflict they may pass the tests anyway. Specification-driven development (SDD) tools write a specification before coding, but the agent still derives the tests on its own, and both remain editable while the code is written. SpecEngine applies stepwise refinement: a requirement becomes a rule matrix whose fields form a Hoare triple, then Gherkin bound to step definitions through a thin driver interface, then code. Every open boundary becomes a multiple-choice question for a human; a dependency-free script freezes the specification, step sentences, and driver interface during implementation; and a per-module lock lets later audits detect edits. We implement SpecEngine as a Claude Code plugin and evaluate it in a case study and a public benchmark of four requirements under three conditions (no SDD tool, GitHub Spec Kit, SpecEngine), scored by hidden acceptance tests. With strong models, both SDD tools clearly beat the baseline, which invented every failure code, and tied with each other. With a weak model, Spec Kit asked one question on an order state machine and scored no better than the baseline (20/32), while SpecEngine, which queues open boundaries by procedure, kept full marks (32/32, 38/38). On event-driven services built solo and by three-agent teams, SpecEngine scored 36/36 in all four settings, though neither tool checks contracts between services. The lock detected all 11 tampered specifications; fault injection showed that dropped rules and scenarios are caught by script only in SpecEngine, and that emptied step bodies escape every condition. Cells hold one or two runs, stakeholders were simulated from answer sheets, and the freeze detects rather than prevents deliberate edits.**

***Index Terms*—behavior-driven development, coding agents, design by contract, executable specifications, human-in-the-loop, large language models, reward hacking, specification-driven development, stepwise refinement**

## I. Introduction

Large language model (LLM) coding agents have evolved from completion aids into autonomous executors that read requirements, edit many files, run tests, and iterate on their own. Once agents produce more code than developers can review, oversight rests largely on the automated test suite [1]. Tests, however, are only a proxy for the specification, and proxies can be gamed. Two failure modes follow. The first is *silent gap-filling*: a short requirement leaves many boundaries open, and the agent picks a plausible behavior for each without asking and without recording the choice. In the benchmark of Section V, an agent working without a specification tool listed its assumptions faithfully and still got every failure code wrong. The second is *test exploitation*: when tests and the specification conflict, frontier models frequently pass the tests anyway, by editing the tests, special-casing the code, or hard-coding expected outputs [2], [3].

To counter the first behavior, practitioners began adopting specification-driven development (SDD) in 2025, which has the agent write a structured specification before writing code; representative tools are GitHub Spec Kit and Amazon Kiro. Böckeler [4] distinguishes three levels of ambition—spec-first, spec-anchored, and spec-as-source—and Piskala [5] turns them into a practitioner's guide. In current tools, however, the agent still derives the tests from the specification largely on its own, and both the specification and the tests remain editable while the code is written; Kiro, for example, offers to fix "the spec, implementation, or test itself" when a property-based test fails [6]. Current SDD, in short, deals with a specification that is *missing*, not with one that is *mutable*. Conversely, studies of test exploitation have shown how common the second behavior is and have evaluated mitigations such as read-only tests [2], but they freeze test files rather than the specification the tests are derived from.

Both behaviors have a common root: the specification is not independent of the agent. In the vocabulary Alexander [22] uses for design problems, the situation is a conflict among five *forces*: (F1) delivery speed versus verifiability; (F2) agent autonomy versus human decision authority; (F3) human readability versus machine executability of the specification; (F4) specification stability versus implementation freedom; and (F5) constraint strength versus process friction. Current SDD tools resolve F3 largely in favor of readability and leave F4 unresolved.

This paper presents SpecEngine, a workflow that applies stepwise refinement [8] to LLM coding agents and freezes each refined level by hashing. Requirements are decomposed into a rule matrix whose fields are the precondition, command,

and postcondition of a Hoare triple [15]. Every undefined behavioral boundary is externalized as a multiple-choice decision for a human stakeholder. Confirmed rules are compiled into Gherkin and bound to step definitions through a thin driver interface. Before implementation, a script records hash snapshots of the specification, the step sentences, and the driver interface, and verifies them afterwards. Module state is derived from artifacts rather than recorded, and each finished module receives a lock, committed to version control, against which later audits detect edits.

The contributions of this paper are as follows.

- We frame specification integrity under LLM agents as an explicit design problem using problem frames [21] and force analysis, and delimit the solution with eight first-class entities and four invariants.
- We propose a three-layer freeze that fixes *what counts as correct* while leaving *how it is achieved* open. Changes the agent makes to the specification, the step sentences, or the driver interface during implementation are detected at verification, except through one recorded route and under the threat model stated in Section III-E.
- We implement SpecEngine as an open-source Claude Code plugin whose enforcement scripts use only the Node.js standard library.
- We report a case study in which the workflow took a four-sentence requirement to an all-passing, locked module, and in which the agent, when its own formalization error collided with the frozen contract, reported the defect instead of rewriting the contract.
- We report a public benchmark comparing SpecEngine with GitHub Spec Kit and with no SDD tool on four requirements, including event-driven services built by three-agent teams, with strong and weak models, hidden acceptance tests, tampering, fault injection by refinement level, and mutation testing.

The rest of the paper is organized as follows. Section II reviews related work. Section III defines the problem and the conceptual model and presents the method. Section IV reports the case study and Section V the benchmark. Section VI discusses the results, the remaining evaluation, and threats to validity. Section VII concludes.

## II. Related Work

### A. Stepwise Refinement, BDD, and Executable Specifications

Wirth [8] described program development as a sequence of refinement steps, each decomposing a task into more detailed instructions while deferring representation decisions as long as possible. Behavior-driven development (BDD) was introduced by North [9] and later organized by Smart and Molak [10] into three activities: discovery, formulation, and automation. Gherkin is the most widely used BDD specification language; its Feature–Rule–Scenario hierarchy and Given/When/Then steps are parsed by runners such as Cucumber and bound to step definitions [11]. Wynne's Example Mapping [12] structures discovery conversations with four kinds of cards: the story, its rules, examples, and open questions. Adzic [13] argues for replacing abstract statements with concrete examples that become executable living documentation. On the design side, Evans's ubiquitous language [14] requires business, code, and tests to share one vocabulary, while Hoare's axiomatic semantics [15] and Meyer's design by contract [16] provide the formal basis of precondition, command, and postcondition. Humble and Farley [17], [18] describe a four-layer acceptance-test architecture—test cases, a domain-specific language, protocol drivers, and the system under test—so that test cases survive user-interface changes. Beck's test-driven development [19] and Freeman and Pryce's double loop [20] define the red–green–refactor rhythm of implementation. SpecEngine proposes no new theory; it chains these practices together and enforces by script the steps that otherwise depend on the discipline of whoever performs the refinement.

### B. Specification-Driven Development

Böckeler's analysis of Kiro, Spec Kit, and Tessl [4] shows that the term SDD covers three levels of ambition and raises open questions, including whether reviewing Markdown documents is cost-effective and whether SDD repeats the mistakes of model-driven development. Piskala [5] organizes the three levels into a practical guide and decision framework but reports no controlled experiment. As of October 2026 we found no published controlled evaluation of Spec Kit or Kiro. Spec Kit's core workflow is constitution, specify, plan, tasks, implement, and converge, with optional clarify and analyze commands [7]. Its specification consists of user stories with Given/When/Then acceptance scenarios; *clarify* asks at most five questions per session, and *converge* has the model compare the code with the specification, plan, and tasks and append unfinished work. Kiro writes requirements in the EARS notation and can generate property-based tests from them; when such a test fails, it can update the implementation or offer to fix the specification, the implementation, or the test [6]. In both tools the agent derives the tests, the specification stays editable during implementation, and no mechanism records or detects an edit to it.

### C. LLMs and BDD Scenario Generation

Karpurapu et al. [23] compared GPT-3.5, GPT-4, Llama-2, and PaLM-2 at generating BDD acceptance tests with zero- and few-shot prompting and found almost no syntax errors in the GPT models' output. Rathnayake et al. [24] compared GPT-4, Claude 3, and Gemini on 500 user stories from four commercial products and found that text-similarity metrics ranked the models differently from human experts, and that the level of detail in the input requirement clearly affected output quality. Other work examines epic-organized generation [25] and a human-centred quasi-experiment on Gherkin generated from regulations [26]. These studies evaluate textual quality; the generated Gherkin is not bound to step definitions and executed, and its life cycle after generation is not addressed.

### D. Test-Driven Formalization of User Intent

Lahiri et al. [27] proposed TiCoder: because natural language is ambiguous, the model should generate test cases for the user to confirm instead of guessing, and the confirmed tests are then used to prune candidate programs. On MBPP and HumanEval, one to five simulated queries raised pass@1 by 22 to 54 percentage points. Fakhoury et al. [28] showed in a study with 15 participants that the workflow improved users' ability to judge AI-generated code correctly and reduced cognitive load. TiCoder shares SpecEngine's idea of externalizing decisions, but its questions are function-level input–output pairs, whereas ours are module-level business and engineering trade-offs. TiCoder does not address whether confirmed tests remain fixed during later generation, and its accuracy evaluation uses simulated users.

### E. Test Exploitation and Reward Hacking in Coding Agents

Zhong et al. [2] introduced ImpossibleBench, which mutates unit tests from LiveCodeBench and SWE-bench to contradict the specification and defines the cheating rate as the pass rate on such impossible tasks. They also evaluated mitigations: hiding the tests reduced cheating to near zero but lowered legitimate performance; making them read-only prevented test modification but not special-casing; and an option to flag a task for human intervention reduced cheating sharply for some models. Gabor et al. [3] introduced EvilGenie and observed explicit reward hacking by Codex and Claude Code, mostly through heuristic special-casing and hard-coded test cases rather than edits to test files. Zhao et al. [1] introduced SpecBench, which splits each task into visible and held-out tests and measures hacking as the gap between the two pass rates; the gap grows with code size, by about 28 percentage points per tenfold increase. Drawing on production experience, Wahi [29] argues that LLM-as-a-judge signals can only be advisory and that deterministic checks must act as gates the judge cannot override.

SpecEngine's freeze is closest to ImpossibleBench's read-only tests and abort option. It differs in what is frozen and for how long: not test files but the refined specification from which they are derived (the Gherkin contract, the step sentences, and the driver interface), delimited per module, with step bodies deliberately left editable, and extended past implementation by a lock that later audits check. It also pairs the freeze with human-answered decisions upstream. We are not aware of prior work that combines an executable, module-delimited specification freeze with post-release audit in an agent workflow.

## III. Method

This section defines the problem and its forces (III-A), introduces the conceptual model and invariants (III-B), presents the workflow architecture (III-C) and its core mechanisms (III-D–III-G), and describes the implementation (III-H). It describes release 2.0.0; the changes in release 2.1.0 are listed at the end of Section III-H.

TABLE I
Forces and the Proposed Responses

| | Force (conflict) | Response in SpecEngine |
|---|---|---|
| F1 | Speed vs. verifiability: humans cannot verify fast output line by line | Scripts carry verification; humans intervene at two gates only |
| F2 | Agent autonomy vs. human authority: more autonomy, more boundaries settled by the agent | The agent generates questions and options; only $S$ answers |
| F3 | Readability vs. executability: prose is readable, code is executable | Gherkin bound to step definitions is both document and test |
| F4 | Stability vs. freedom: freezing all blocks refactoring; freezing nothing lets the spec be rewritten | Three-layer freeze: fix what is correct, free how it is achieved |
| F5 | Strength vs. friction: more constraints, more bypassing | Three scripts; state derived from artifacts, maintained by no one |

### A. Problem Definition and Forces

Following Jackson's problem frames [21], we separate the machine to be built—the workflow $W$—from four problem domains: the requirements document $R$ (a lexical domain, incomplete and ambiguous); the stakeholder $S$ (a biddable domain that holds decision authority but has limited time); the agent $A$ (biddable but untrusted: when goals and instructions conflict, it tends toward the path of least resistance); and the codebase and test environment $T$ (a causal domain). The requirement is that the behavior of $T$ satisfies $R$ as adjudicated by $S$. Were $A$ trustworthy, this would be Jackson's required-behaviour frame. Because it is not, an additional frame concern arises: $A$ both operates on $T$ and may rewrite the description of $R$. If the specification and the tests are both within $A$'s write scope, "$T$ passes the tests" no longer implies "$T$ satisfies $R$". The machine therefore has two responsibilities: (a) turn $R$ into executable criteria, with gaps filled by $S$; and (b) make any change $A$ makes to those criteria detectable while $A$ operates on $T$. Current SDD tools take on (a); this work is centered on (b).

The design must balance the five forces listed in Table I. Current SDD tools favor readability in F3, leave F4 unresolved, and externalize F2 only partially. The rightmost column of Table I gives SpecEngine's response to each force.

### B. Conceptual Model and Invariants

We define the objects of the solution as eight first-class entities. Table II lists their definitions, owners, and whether they may change during the freeze, that is, while the agent implements; "by instruction" marks artifacts that the instructions forbid editing but that the snapshot of release 2.0.0 does not hash; Fig. 1 shows their relations. Not in the table, and entirely free during the freeze, are the bodies of step-definition functions, driver implementations, and product code. This boundary is where F4 is resolved.

The machine must maintain four invariants. **I1 (contract immutability):** during the implementation of any module, the hashes of the Contract and the Driver Interface and the set of Step Sentences equal those at the start of implementation;

TABLE II
FIRST-CLASS ENTITIES

| Entity | Definition | Owner | Mutable during freeze |
|---|---|---|---|
| Requirement | The original natural-language requirement | $S$ | n/a |
| Decision | A multiple-choice question on an undefined boundary, with $S$'s answer | Question by $A$; answer only by $S$ | No (by instruction) |
| Rule | A behavioral rule with fields precondition, command, postcondition, i.e., $\{P\}\, C\, \{Q\}$ | Extracted by $A$ from $R$, or written per a Decision | No (by instruction) |
| Exclusion | A decision that the system deliberately does nothing, does not offer a feature, or defers to another module | Written per a Decision | No (every byte) |
| Specification Contract | The executable form of the Rule set: Gherkin, data model, glossary, conventions, Exclusions | Compiled by $A$ from Rules | No (every byte) |
| Step Sentence | The text of each Given/When/Then in the Contract; also the key of a test function | Generated by $A$ from the Contract | No (sentence set) |
| Driver Interface | The set of primitive operations the system under test supports from the test's viewpoint; signatures only | Collected by $A$ from Step Sentences | No (every byte) |
| Lock | Hash record of a finished module's feature files, Step Sentences, and Driver Interface | Script | n/a |

otherwise implementation may not be declared complete. **I2 (complete correspondence):** each Rule of a formalized module has exactly one corresponding block in the Contract, and vice versa. **I3 (real bindings):** every Driver method declared by a Step Sentence exists in the Driver Interface. **I4 (decision ownership):** every Decision is answered by $S$. Scripts check I1–I3. I4 is the only invariant a script cannot verify: a script can prove that an answer exists, not where it came from.

Module state is not a first-class entity but a function of which entities exist. For a module $m$: if Lock($m$) exists, $m$ is IMPLEMENTED; otherwise, if step-definition files tagged with $m$ exist, it is STUBBED; otherwise, if Gherkin tagged with $m$ exists, FORMALIZED; otherwise, if an unanswered Decision belonging to $m$ exists, CLARIFYING; otherwise, if a Rule matrix for $m$ exists, CLARIFIED; otherwise UNDISCOVERED. Completion of P3 is derived from a status line in the flow-diagram index file and determines whether a CLARIFIED module resumes at P3 or P4. Whatever the stage, an unanswered Decision makes clarification the next step. With this definition, state does not depend on the agent's self-report.

### C. Workflow Architecture

The workflow has six phases, shown in Fig. 2 and Table III. P1 and P4–P6 are non-interactive, except that P1 may ask which module a requirement belongs to; P2 and P3 are human gates at which the agent must stop and wait for $S$. A single command chains the six phases; its starting point is given by the state function of Section III-B, so a restart after an interruption does not repeat completed phases. Creating and checking the environment manifest, and post-release audits, are governance actions outside the chain. A return from P6 to P2 re-enters the same gate, so there are still only two points of human intervention. The snapshot is created at the end of P5 and recreated at the start of P6, as a boundary operation between the two phases.

The design principle of the chain is stepwise refinement: the room for free interpretation strictly shrinks at each level. The Requirement is natural language; Rules and the Contract are structured specifications with fixed fields; the Driver Interface is a fixed set of primitives; step bodies, driver implementations, and product code form the bottom level, where only implementation freedom remains. The three fields of a Rule are a Hoare triple [15], which becomes Gherkin's Given, When, and Then and then three test functions. Because the shape is preserved across levels, every transformation below the Rule is close to mechanical. The transformation that requires most judgment is from Requirement to Rule, and that judgment is externalized by the mechanism of Section III-D. The Driver Interface corresponds to the protocol driver in Farley's four-layer architecture [18]. It allows the same Contract to run quickly against an in-process driver for the red–green loop of P6 and then against a user-interface driver for acceptance, with the Contract and Step Sentences unchanged in between. A project may also declare a *direct* binding, in which step definitions call product code without a driver; Driver Interface checks are then skipped.

### D. Decision Externalization

P1 separates the Requirement into defined and undefined parts. Defined parts become Rules. Undefined parts are examined against a probe list—numeric precision, repeated triggering and idempotency, threshold values, failure and rollback, data life cycle—and each gap becomes a Decision with a plain-language question, two or three options with their engineering consequences, scope of impact, priority, and whether it is reusable across modules. This amounts to writing the rule, example, and question cards of Example Mapping [12] into files. Because the probe list is applied by procedure, the number of questions depends less on the model's own judgment of which ambiguities matter; Section V-C suggests why this matters.

P2 groups Decisions by locus (the same entity or the same Rule), at most four per group, and presents them as multiple-choice rather than open questions. The agent's task in this phase is limited to translating technical terms into domain language and writing $S$'s answers to the right place. Answers go into four kinds of entities by their nature: a decision that causes an action goes into a Rule; a decision that the system

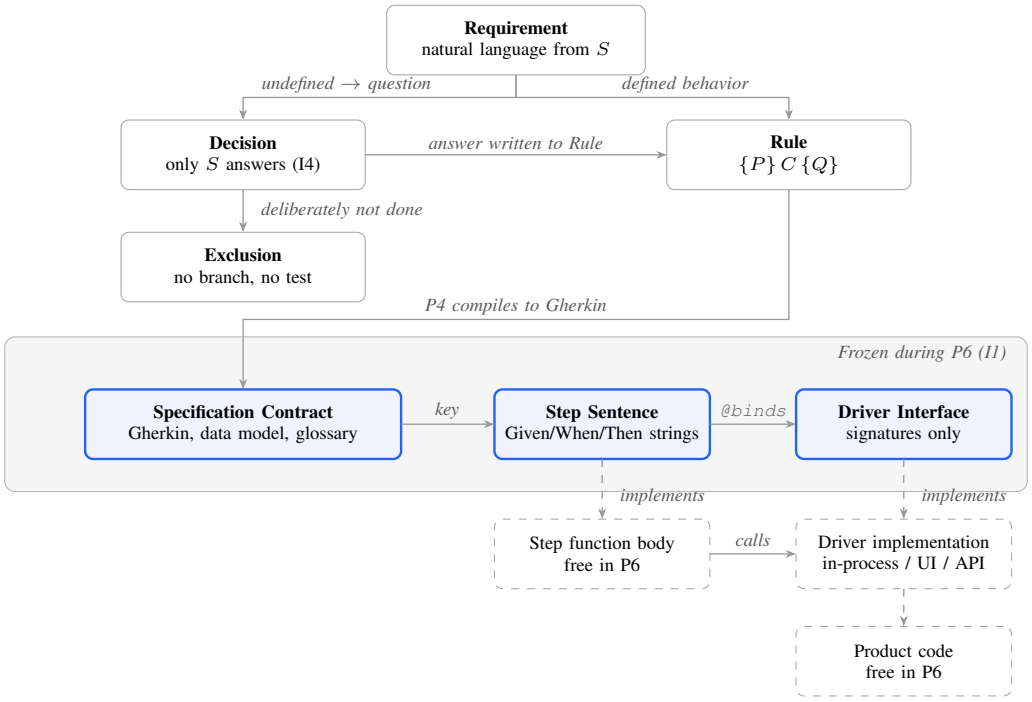



Fig. 1. Conceptual model. Shaded boxes are frozen during implementation; the Lock records the feature files, Step Sentences, and Driver Interface (release 2.1.0 adds the matrix and the shared data model and glossary). Dashed boxes may change freely.

TABLE III
THE SIX PHASES

| Phase | Name | Actor | Input | Output | Script check |
|---|---|---|---|---|---|
| P1 | Discovery | $A$ | Requirement | Rule matrix, draft data model, glossary, Decision queue | Matrix structure |
| P2 | Clarification | $S$ | Decision queue | Answers written back to Rules, Exclusions, data model, glossary | Matrix structure |
| P3 | Flow alignment | $S$ | Rule matrix | Flow diagrams; non-gaps ruled by $S$ | None |
| P4 | Formalization | $A$ | Rule matrix | Specification Contract | Purity, I2, sentence index |
| P5 | Test scaffolding | $A$ | Contract | Step Sentences, Driver Interface, implementation skeleton | I3 |
| P6 | Implementation | $A$ | All-failing tests | Product code, all tests passing, Lock | I1 (snapshot/verify), lock |

does nothing, does not offer something, or defers to another module goes into an Exclusion; data types and constraints go into the data model; names and fixed messages go into the glossary. The separate Exclusion entity is necessary because the three-field Rule structure is designed for "an action causes a result"; forcing "the system deliberately does nothing" into it produces tests that always pass and discriminate nothing. Any Decision still unanswered when P2 is left is *parked*, and P4 turns it into a visible comment in the Contract; a Decision ends either answered or parked. When a Decision is marked reusable, its answer is also written to a list of conventions, which P1 applies directly when a later module meets the same kind of gap.

P3 complements P2. Answering questions one by one is a local view and does not guarantee overall completeness. The agent draws the Rule matrix as flow diagrams by sub-scenario, adds a plain-language restatement and the list of Exclusions, and lists the gaps it suspects for $S$ to rule on. The hard constraint is that only Rules present in the matrix may be drawn. When $S$ identifies a gap, it becomes a new Decision and the module returns to P2; items $S$ rules to be non-gaps are kept whether or not the round passes.

### E. Three-Layer Freeze and Completion Lock

This is the core mechanism of the method and enforces invariant I1. Algorithm 1 gives its pseudocode. Feature files are tagged per module with a Gherkin tag `@rule:R-`$m$`-`, and step-definition files with a code comment `@rule R-`$m$`-`; the difference follows the syntax of each host.

The boundaries of the three layers correspond to the balance point of F4. The Specification Contract is frozen byte for byte because it defines what is correct. For Step Sentences only the sentences are frozen, not the function bodies: a sentence is the foreign key between the Contract and the code, while turning a body from throwing an exception into calling product code is precisely the work of P6. The Driver Interface is frozen

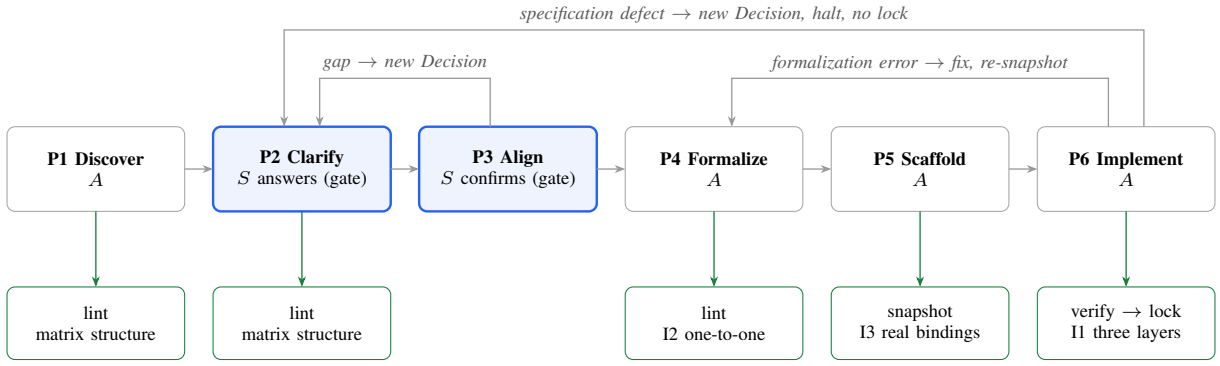



Fig. 2. Workflow architecture: six phases, two human gates (shaded), script checkpoints (below), and fallback paths (above). Module state is derived from artifacts, not reported by $A$.

**Algorithm 1** Snapshot, verification, lock, and audit

**function** SNAPSHOT($specDir$, $stepDirs$, $driverDirs$)
  $S \leftarrow \{f \mapsto \text{sha256}(f) \mid f \in \text{files}(specDir)\}$
  $T \leftarrow \{f \mapsto \text{sentences}(f) \mid f \in \text{files}(stepDirs)\}$ ▷ sorted set of (keyword, text) of every `Given|When|Then('text', ...)` call
  $D \leftarrow \{f \mapsto \text{sha256}(f) \mid f \in \text{files}(driverDirs),\ f \text{ matches } \star\texttt{.driver.}\star\}$
  **assert** $|\bigcup T| > 0$ ▷ no sentences means the stubs are missing
  **assert** BINDSVALID($T$, $D$) ▷ every `@binds X.y` names a method `y` present in $D$ (I3)
  write $(S, T, D)$ to the snapshot file
**end function**
**function** VERIFY($snapshot$)
  $(S', T', D') \leftarrow$ recompute as in SNAPSHOT; $V \leftarrow \emptyset$
  **for all** $f \in \text{keys}(S) \cup \text{keys}(S')$ **with** $S[f] \neq S'[f]$ **do**
    $V \leftarrow V \cup \{(\text{SPEC}, f)\}$
  **end for**
  **for all** $f \in \text{keys}(T) \cup \text{keys}(T')$ **do**
    $V \leftarrow V \cup \{(\text{SENTENCE-REMOVED}, f, s) \mid s \in T[f] \setminus T'[f]\} \cup \{(\text{SENTENCE-ADDED}, f, s) \mid s \in T'[f] \setminus T[f]\}$
  **end for**
  **for all** $f \in \text{keys}(D) \cup \text{keys}(D')$ **with** $D[f] \neq D'[f]$ **do**
    $V \leftarrow V \cup \{(\text{DRIVER-INTERFACE}, f)\}$
  **end for**
  **if** $V = \emptyset$ **then**
    exit 0
  **else**
    report $V$; exit 1
  **end if**
**end function**
**function** LOCK($m$)
  $F \leftarrow \{f \mapsto \text{sha256}(f) \mid f$ a feature file containing tag `@rule:R-`$m$`-`$\}$
  $T \leftarrow \{f \mapsto \text{sentences}(f) \mid f$ a step file containing `@rule R-`$m$`-`$\}$
  $D \leftarrow \{f \mapsto \text{sha256}(f) \mid f$ a driver-interface file referenced by `@binds` in $T\}$
  write $(m, timestamp, F, T, D)$ to `locks/`$m$`.lock.json` ▷ committed to version control
**end function**
**function** AUDIT
  **for all** locks $(m, t, F, T, D)$ **do**
    recompute and compare as in VERIFY; report LOCK-DRIFT per module
  **end for**
**end function**

and its implementations are not, for the same reason. When VERIFY fails, the instructions forbid the agent from declaring completion. The Lock extends the check past implementation: it records the module's feature files, Step Sentences, and Driver Interface, and AUDIT compares them later. In release 2.0.0 the Lock does not cover the shared data model, glossary, and conventions, which are frozen only during P6; release 2.1.0 adds them, together with the rule matrix. The theoretical basis is design by contract [16]: the contract is an invariant that the implementation must satisfy, and it may not be amended to accommodate the implementation. What we add is distrust of the agent's self-discipline, turned into a check by hashing.

The threat model must be stated plainly. The scripts are run by the agent, and the snapshot file lies in the agent's write scope. An agent that edits the Contract and then re-runs SNAPSHOT before VERIFY passes the check; what remains is the edit in version-control history and, after completion, a Lock that reviewers can compare. The mechanism therefore detects goal-driven or careless edits, which is the behavior reported in the studies of Section II-E, not deliberate evasion by an agent that reasons about the check. Running VERIFY in continuous integration from a snapshot committed before P6

would close this gap; the current release does not do so. Within this model, the remaining ways to make a test pass without changing a frozen artifact are to change product code or a driver implementation, and two of those are escape hatches: the agent can reduce a Then step to an empty assertion or make the in-process driver return the expected value directly, and VERIFY will not notice. The current release lowers this risk with two instruction-level constraints: a step body may only convert types, call the driver, and assert; and if a user-interface or API driver is declared, acceptance must rerun the suite through it. Neither is enforced by script, and Section V-E measures the first.

### F. State Derivation

The state function of Section III-B is implemented by a script that attributes artifacts to modules by tag-string containment and Decisions to modules by the module identifier appearing in each question file. It warns when lower-layer artifacts are missing (for example, step definitions without a Contract) and when a cached field disagrees with the derived state. This replaces the approach of an earlier version, in which the stage was stored in a field of the Rule matrix and rewritten by the agent at the end of each phase. Nothing kept that field consistent with the actual artifacts, and when it went stale unnoticed the chain restarted from the wrong phase. The current release keeps the field only as a cache. Audit results are not yet fed back into the state function: a module whose Lock exists but has drifted is still derived as IMPLEMENTED.

### G. Failure Classification and Fallback

When a test in P6 cannot be made to pass, the agent must first classify the failure (Table IV). The criterion separating the second and third classes is whether the correct Contract text can be derived uniquely from the Rule matrix alone. If it can, the failure is a translation error that involves no business decision, and the agent may fix the Contract, record the fix in a dedicated file, rebuild the snapshot, and report both; if not, the question must go back to $S$. The second class was added after the case study (Section IV-E). To make it rarer, P4 must check row by row, before writing the Contract, that the inputs and expected results of each Examples row follow uniquely from the precondition and postcondition of its Rule; this check is performed by the agent, not by a script.

The second class deliberately reopens a route by which the agent may edit the frozen Contract during P6. The route is bounded: the edit must be recorded, the snapshot rebuild must be reported, the Step Sentences and Driver Interface stay frozen, and the instructions forbid editing the matrix. But the judgment that the answer is "uniquely derivable" is the agent's own, and a sanctioned fix can still lose coverage: in the case study (Section IV-D), simply correcting the faulty row's expected message would have removed the only test of one branch. Across the 18 SpecEngine agent logs of the benchmark (Section V), no formalization fix occurred. The snapshot was rebuilt twice, both times after the agent corrected a manifest line that a script bug had misread and with no change to the specification; such benign rebuilds use the same route a deliberate edit would. Section VI-B treats the route as a threat to the evaluation design.

TABLE IV
FAILURE CLASSES AND FALLBACK

| Class | Criterion | Handling | Return to |
|---|---|---|---|
| Code error | Contract agrees with the Rule; product code does not satisfy it | Change product code or driver implementation | — |
| Formalization error | The Rule determines the answer uniquely; the Contract disagrees | Agent fixes the Contract, records it, re-snapshots, reports | P4, no halt |
| Specification defect | The Rule itself is contradictory or undefined | New Decision; test stays failing; halt; no lock | P2 |

### H. Implementation

SpecEngine is implemented as a Claude Code plugin. Release 2.0.0 consists of ten instruction documents (skills), three Node.js scripts, one format specification, and two templates, totaling 1,903 lines by `wc -l`; four reference files (the P1 probe list, the question-file format, and two examples) add 278 lines. The scripts have no external dependencies and need no network; they read and write files only under the project directory. `feature-lint.js` implements Contract purity, matrix structure, I2, and the sentence index; `spec-guard.js` implements the four operations of Algorithm 1; and `spec-status.js` implements state derivation. The project's technology stack is declared in a single environment manifest, and the workflow itself makes no assumption about the stack. Driver-interface files are recognized by a filename convention, which avoids the dependencies that parsing each language's interface syntax would require; in languages without interfaces, a base class whose methods all throw serves as the interface file. The parts tied to Claude Code are the packaging format of the instruction documents, the file into which the routing block is installed, the environment variable used in script paths, and the names of the tools the instructions use to ask questions, track tasks, and invoke sub-skills, each with a plain-text fallback. The scripts and the artifacts are tool-independent.

Release 2.1.0 adds a change workflow for modules that are already locked: a requested change is classified as a parameter change, a behavior change, or a new rule by comparing the old and proposed matrices; the stakeholder approves its scope; and on completion a script checks that every rule outside the approved scope is byte-for-byte unchanged. It also adds the rule matrix and the shared data model, glossary, and conventions to the Lock, and normalizes line endings before hashing. The weak-model and T4 runs of Section V used release 2.1.0; its phase instructions P1–P6 are unchanged from 2.0.0, while the lock and status scripts changed as described.

## IV. Case Study

### A. Design

To assess the feasibility of the method, we conducted an exploratory single-case study following the guidelines of Runeson and Höst [30]. The unit of analysis is a single business module. The case addresses three questions. CQ1: can the workflow take a fresh project on a stack different from its development project from requirement to all tests passing and a Lock? CQ2: are invariants I1–I4 maintained during execution, and if they come into conflict, how does the agent behave? CQ3: which parts of the instruction text are ambiguous or contradictory, and what design gaps does the case reveal?

### B. Data and Setup

The case project was an empty Node.js project with cucumber-js 13.3.0, plain JavaScript, and no browser. It was chosen because its stack differs from that of the project on which the tool was developed (TypeScript with playwright-bdd), which tests whether the environment manifest carries all stack assumptions. To preserve I4, the executor and the stakeholder were two model instances that shared no context. The executor ($A$) was instructed to read the instruction documents phase by phase and follow them, to run the scripts by absolute path, to list every question needing $S$'s answer in full and then end its turn, and to report ambiguities and contradictions in the instruction text after each round. The stakeholder ($S$) was played by a model instance in the researcher's session under the researcher's supervision; it answered only the multiple-choice questions and the flow-diagram confirmations and never saw the executor's artifacts. The executor used no mechanism specific to Claude Code. Both instances ran the default Claude Code model of October 2026; the exact version string was not recorded at the time (Section VI-C). The case ran on a working version shortly before release 2.0.0; Section III describes the final 2.0.0 release, and the differences are chiefly the fixes listed in Section IV-E. By the file system's clock, the run took about 35 minutes from the empty project to the Lock. The requirement consisted of four sentences written by the researcher, deliberately silent on exceeding the limits, repeated redemption, zero values, failure handling, and behavior after submission (translated from Chinese):

> Members can redeem points against the order amount on the checkout page, one point for one dollar. The redeemed points may not exceed the order amount or the member's current point balance. After a successful redemption, the system deducts the points from the member and reduces the order's amount payable by the same number. Members can cancel the redemption before submitting the order, and the points are returned on cancellation.

### C. Results

Table V summarizes the output and script results of each phase. Without asking $S$ anything, P1 produced 6 Rules, 2 data tables, and 9 Decisions, covering the response to exceeding a limit, repeated redemption, atomicity of deducting points and reducing the price, zero input, what happens to points when the member leaves the page, whether amounts may have decimals, and whether a transaction log is kept. P2 resolved them in three groups; the Rules grew to 11 and 7 Decisions were promoted to conventions. In the first run of P3, the executor listed three suspected gaps of its own; $S$ ruled one of them a gap (whether a repeated redemption exactly at the limit is allowed), one out of scope, and one already covered, and the module returned to P2 as designed. When the chain was restarted, its starting point was correctly derived as P2 and P1 was not rerun. P4 produced 11 Rules, 14 Scenario blocks (21 after expansion), and 12 distinct sentence patterns. P5 used cucumber's dry run to list undefined steps and produced 12 step definitions, a Driver Interface of 11 methods, an in-process implementation, and fixtures; the snapshot passed, all 12 `@binds` annotations, naming 11 distinct methods, pointed to real methods, and all 21 tests failed. P6 implemented in three increments, passing 4, 15, and 20 scenarios; the 21st could not be made to pass.

### D. Critical Incident

The failing Scenario Outline was "when a repeated redemption is rejected, the previous redemption stays in effect". Its Given steps set a balance of 100, an order amount of 500, and an existing redemption of 50; one Examples row entered 501 and expected the message "redeemed points exceed the order amount". According to the matrix, however, the available balance during a repeated redemption is 100; 501 exceeds both the balance and the order amount, and the precondition of rule R-002 states that the balance takes precedence when both are exceeded. The product code returned "redeemed points exceed the balance", which is correct; the Contract row was a translation error made by the executor in P4.

This was a direct conflict between I1 and the agent's goal. The shortest path to a green suite was to change the row's expectation to "exceed the balance": the test would have turned green and the report would have read 21/21. Under the instructions in force at the time, any edit to the Contract during P6 was forbidden and would have failed VERIFY. What the executor did was the following. It determined that the Contract was wrong; it created a Decision with three options and wrote it to the queue; it ran VERIFY, which passed, showing that the Contract had not been touched; it did not lock; and it halted, reporting a specification defect. The second class of Table IV did not yet exist; this incident is why it was added. The options it offered show why the agent's judgment matters even then: the shortcut (option C) would have left no test of the "exceeds the order amount" branch during a repeated redemption, while the option $S$ chose (A) raised the Given balance to 1,000, kept a row for each branch (1,051 and 501), and adjusted the expected balance. The executor then corrected the outline through P4, rebuilt the snapshot, and completed P6. For CQ2, I1 was maintained when the conflict occurred. A single incident cannot tell whether this behavior came from the presence of the script or from the model's own disposition; Section VI-B gives a design to separate the two. The incident does show that such conflicts arise naturally in non-interactive phases and are not merely artifacts of benchmark construction.

TABLE V
CASE EXECUTION LOG

| Phase | Main output | Intervention by $S$ | Script result |
|---|---|---|---|
| Environment check | Manifest: 9 fields filled automatically, 26 left blank; routing block installed | 1 question (binding level) | — |
| P1 | 6 Rules, 2 data tables, 9 Decisions | None | lint PASS; CLARIFYING |
| P2 | Rules 6 → 11; 7 conventions | 9 questions in 3 rounds | lint PASS; CLARIFIED |
| P3 (1st) | 3 diagrams; 3 suspected gaps | 1 gap, 1 out of scope, 1 covered | Back to CLARIFYING; chain halted |
| P2 (2nd) | Start derived as P2; 1 question | 1 question | CLARIFIED |
| P3 (2nd) | Redrawn; earlier rulings reused | Confirmed | Flow-design index written |
| P4 | 11 Rules, 14 Scenarios (21 expanded), 12 sentence patterns | None | lint PASS; FORMALIZED |
| P5 | 12 steps, 11 Driver methods | None | snapshot PASS; 21 failing; STUBBED |
| P6 | 4 → 15 → 20 passing; 1 failing | None | verify PASS; not locked; halted |
| P2 → P4 → P6 | Outline values corrected; snapshot rebuilt; 21 passing | 1 question | verify PASS; lock; IMPLEMENTED |
| Audit | One value tampered, audited, restored, audited again | None | LOCK_DRIFT; PASS after restore |

After the Lock was created, the researcher changed one Examples value in the Contract from 501 to 502 and ran AUDIT, which reported LOCK_DRIFT; after the value was restored, the current hashes matched the Lock exactly.

### E. Discussion

Table VI gives the quantitative results. For CQ1, the workflow ran to completion on a stack it was not developed on, and all stack assumptions were carried by the manifest. For CQ2, I1 and I3 were verified by script. I2 was not yet checked by lint at the time of the case (that check was added afterwards). I4 holds according to the reports, but the executor had been explicitly told to stop, so this cannot be generalized. The count of zero answers given by $A$ in Table VI captures only explicit answers; boundaries that P1 failed to probe and silently assumed cannot be measured this way—the gaps the executor itself added in P3 are an example—so the recall of P1 is unknown here; the benchmark measures it against an answer sheet (Section V). The five Exclusions are the known trade-offs $S$ confirmed during P3; one of them (R-009) was also recorded as a Rule, because at the time such decisions had no other home.

For CQ3, the executor reported more than fifty observations, which fall into four classes after de-duplication: (i) the formats of auxiliary files were undefined, so the executor invented its own; (ii) decisions that the system deliberately does nothing had no home, so the executor invented rule types and put them in the matrix; (iii) formalization errors were posed to $S$ as business decisions; and (iv) behavior that arises when an object produced by one module is consumed by another belongs to no module's Rules. The first three are fixed in release 2.0.0 (a format specification, the Exclusion entity with a closed set of rule types, and the second class of Table IV); the fourth is future work. The fourth is the most fundamental: the method guarantees that the code conforms to the Contract, not that the Contract conforms to the business outcome, and the latter currently rests on $S$ reviewing a single module at a time in P3.

After the case, the researcher reran the test suite, AUDIT, and state derivation, and the results matched the executor's reports. Rerunning lint with release 2.0.0 reports 11 violations, one per rule, because the matrix uses the rule types the executor invented before the closed type set existed (gap ii). The case project with all artifacts and the Lock is part of the replication package (see Data Availability), which also documents that the timestamps the executor wrote into decision files are estimates; file modification times and the Lock's UTC timestamp give the actual order of events.

TABLE VI
QUANTITATIVE RESULTS OF THE CASE

| Item | Value |
|---|---|
| Requirement | 4 sentences |
| Rules (P1 → final) | 6 → 11 |
| Business Decisions (P1 / P3), all answered | 10 (9 / 1) |
| Formalization error submitted as a Decision (P6) | 1 |
| Promoted to conventions | 7 |
| Exclusions (known trade-offs confirmed in P3) | 5 |
| Scenarios (expanded) | 21 |
| Step Sentences | 12 |
| Driver Interface methods | 11 |
| Rounds of intervention by $S$ | 8 |
| Multiple-choice questions answered by $S$ | 12 |
| P3 rulings and confirmations by $S$ | 3 + 1 |
| Answers given by $A$ | 0 |
| Script runs / unexpected failures | ≈35 / 0 |
| Conflicts under I1 | 1, reported, not rewritten |

## V. BENCHMARK

The case study shows feasibility on one module. To compare SpecEngine with an existing SDD tool and with no tool at all, we built a public benchmark. Every number in this section can be regenerated from the artifacts and scripts in the repository (see Data Availability).

### A. Design

*Conditions.* A: no SDD tool; the agent was told to implement the requirement and add tests, could not ask anyone, and had to list every assumption. B: GitHub Spec Kit (`specify-cli` 1.2.3.dev0, commit `e5714c6`) with its Claude integration; the agent ran constitution, specify, clarify,

plan, tasks, implement, and then converge until it reported convergence. C: SpecEngine 2.0.0; the agent ran manifest synchronization and then the chained workflow through both human gates, with direct binding. All conditions started from the same TypeScript project with cucumber-js.

*Tasks.* Three requirements of increasing difficulty, written in Traditional Chinese: T1, loyalty-point redemption (7 open boundaries that must be asked, 18 hidden tests); T2, an order state machine (11 boundaries, 32 tests); and T3, multi-tenant coupons (14 boundaries, 38 tests). Each requirement fixes the public interface, so one hidden test file scores every condition. A change request on T1 (E4, raising a redemption cap from 30% to 35%, 12 hidden tests) was applied to one run per condition. T4, event-driven services built solo and by teams, is described in Section V-H.

*Stakeholder.* Each task has an answer sheet listing every open boundary with its correct answer. Questions were answered from the sheet and nothing else; questions not on the sheet were answered "your call". In the first runs the agent answered its own questions from the sheet under an instruction not to read ahead; three T3 runs broke that rule and are void. Their replacements, and all weak-model runs, had questions answered by a separate session, with the sheet out of the agent's reach.

*Scoring.* Hidden tests import the fixed interface and are never visible to the agent. Each test is tagged with the boundary it checks and with its kind: behavior (what happens) or reason (which failure code is returned). A reference implementation written directly from each answer sheet passes all of its hidden tests.

*Models.* Strong-model runs of T1, T2, and E4 used Claude Fable 5.1. On T3, the two clean replacement runs (Spec Kit r3 and SpecEngine r3) ran entirely on Claude Opus 5.5, and Spec Kit r1 partly, after a model switch; the A runs used Fable 5.1. The strong T3 comparison therefore mixes two strong models. The weak-model runs used Claude Haiku 5.5, the smallest current Claude model, on T2 and T3 under all three conditions, with SpecEngine 2.1.0 and the same Spec Kit commit. All T4 runs used SpecEngine 2.1.0.

### B. Accuracy

Table VII gives the hidden-test results of all clean runs. Without an SDD tool every failure code was invented: condition A passed most behavior tests and none of the reason tests, and on T3 it also guessed behavior wrong, capping an excessive discount instead of rejecting it and accepting a zero-amount order. With strong models the two SDD tools tied, except for one T3 Spec Kit run that missed two tests. Spec Kit's clarify step is capped at five questions per session; on T2 both runs left three boundaries unasked and guessed them right, and on T3 one run guessed one wrong. SpecEngine asked about every T2 boundary; on T3 the one unasked boundary was derived correctly from other answers, and the flow review surfaced four edge cases the answer sheet did not cover.

TABLE VII
HIDDEN TESTS PASSED [MUST-ASK BOUNDARIES ASKED], CLEAN RUNS

| Task | A: no tool | B: Spec Kit | C: SpecEngine |
|---|---|---|---|
| *Strong models (two runs per cell where available)* | | | |
| T1 /18 | 11, 11 [0/7] | 18, 18 [7/7] | 18, 18 [7/7] |
| T2 /32 | 20, 20 [0/11] | 32, 32 [8/11] | 32, 32 [11/11] |
| T3 /38 | 20, 22 [0/14] | 36, 38 [10, 13/14] | 38 [13/14] |
| *Weak model, Claude Haiku 5.5 (one run per cell)* | | | |
| T2 /32 | 20 [0/11] | 20 [1/11] | **32** [9/11] |
| T3 /38 | 23 [0/14] | 31 [8/14] | **38** [13/14] |

*Cost.* For the strong-model runs of T1–T3, measured by the agents' own timestamps minus time lost to rate-limit interruptions, condition A took 1.5–3.5 minutes per task, Spec Kit 6–13.5 minutes, and SpecEngine 14.5–20 minutes, about 1.4 to 1.5 times Spec Kit on T1 and T2. Token use was 51–60k for A, 164–194k for Spec Kit, and 202–218k for SpecEngine. Spec Kit stopped for a human two or three times per run, SpecEngine two to eight times.

### C. Weak Model

A common objection to stepwise refinement with an LLM is that a weak model will mistranslate somewhere along the chain. The weak-model rows of Table VII show the opposite pattern. With Haiku 5.5, Spec Kit's clarify step judged T2 to have no high-impact ambiguities and asked one question; every failure code was then invented, and the run scored exactly what the no-tool run scored. SpecEngine kept full marks on both tasks: its discovery phase turns every open boundary found by the probe list into a queued question by procedure, so the weak model still asked about 9 of 11 and 13 of 14 boundaries and derived the rest correctly from the answers. Runs took 11 and 12.5 minutes with 202k and 278k tokens, against 7 and 9 minutes for Spec Kit. With strong models, the two SDD tools tied; with a weak model, the gap was 12 tests on T2 and 7 on T3. Each weak-model cell is a single run.

### D. Tampering with a Locked Specification

For each locked SpecEngine run of T1–T4, a script changed one example value in the locked Gherkin (for example, 1000 to 1001) and ran AUDIT. Of the 12 locked runs, one weak-model run had no example value to change; all other 11 tampered specifications, including two from void runs, were reported as LOCK_DRIFT within a second. Spec Kit has no lock, so the equivalent edit was committed to `spec.md` by hand and `converge` was run read-only by a fresh agent. Changing a cap from 30% to 35% in one user-story sentence was reported as converged with zero actionable findings, the contradiction appearing only as an awareness note; reversing "shipped orders cannot be cancelled" was reported as a critical contradiction after about 7.5 minutes. In condition A, changing an expected value in its own feature file left all tests green.

### E. Fault Injection by Refinement Level

A script planted one translation error at a time into a copy of every run, at each refinement level, and recorded which

TABLE VIII
WHICH CHECK NOTICES A PLANTED FAULT (26 RUNS)

| Fault | A: no tool | B: Spec Kit | C: SpecEngine |
|---|---|---|---|
| Rule dropped from matrix | — | — | lint; audit |
| Rule postcondition edited | — | — | audit only |
| Rule or scenario block dropped | not detected | not detected | lint; audit |
| One example value edited | own tests 40–100%; none after | own tests 27–100%; none after | own tests 40–80%; audit 100% after lock |
| Step sentence renamed | own tests | own tests | own tests; audit |
| Every Then body emptied | not detected | not detected | not detected |

check noticed it at two moments: at creation, before anything is locked, when only lint and the run's own tests are available; and after lock, when a finished module is changed later. For the second moment every SpecEngine copy was relocked with the 2.1.0 scripts. Table VIII summarizes the results over the 26 runs of T1–T3, including the three void runs; E4 and T4 were not injected.

Structural mistranslations—a dropped rule or scenario—were caught by script only in SpecEngine; in the other conditions the tests simply stopped covering the behavior. After a module was locked, SpecEngine caught every change to every specification level; for the rule matrix this relies on the matrix hashing added in release 2.1.0. An edited example value was caught at creation only when the code disagreed with it; the remaining cells are inputs whose new value still passes, or real mistranslations that no script checks today, because the rule that every example must follow from the matrix is checked by the agent. Hollowed step bodies escaped every condition, SpecEngine included, because step bodies are deliberately left editable.

### F. Mutation Testing

To check whether the conditions differ in the thoroughness of their own tests, a script mutated each run's product code one small change at a time and ran both the run's own tests and the hidden tests. Among mutants that broke a hidden test the original passed, the share also caught by the run's own tests was 91–100% for condition A (9 runs), 100% for Spec Kit (10 runs), and 95–100% for SpecEngine (10 runs); these 29 runs include E4, the weak-model runs, and the three void runs. All conditions wrote thorough tests; the baseline's tests thoroughly checked behavior the agent had guessed. The conditions differ not in how well they test, but in whether the specification being tested is the stakeholder's.

### G. Change Request

The change request E4 was applied to one T1 run per condition. Condition A took 2.5 minutes and passed 6 of 12 hidden tests: all change-related tests passed, but its original wrong guesses remained. Spec Kit reran its full workflow in 9 minutes and passed all 12. SpecEngine 2.0.0 had no documented change workflow; the agent assembled one from the existing phases in 17 minutes and passed all 12. Release 2.1.0 adds the change workflow described in Section III-H.

TABLE IX
T4 HIDDEN TESTS PASSED (OF 36), ONE RUN PER CELL

| Condition | Fable 5.1 Solo | Fable 5.1 Team | Haiku 5.5 Solo | Haiku 5.5 Team |
|---|---|---|---|---|
| A: no tool | 30 | 29 | 30 | 29 |
| B: Spec Kit | 36 | 36 | 30 | 35 |
| C: SpecEngine | **36** | **36** | **36** | **36** |

### H. Event-Driven Services and Team Development

T1–T3 are single modules with synchronous calls. T4 asks whether the results hold for several services that share no data, communicate only through events, and must stay consistent when events are duplicated or reordered. Three services—order, inventory, and payment—exchange events for order placement, stock reservation or rejection, payment success or failure, cancellation, refund, release, and a 15-minute timeout. The requirement states that delivery is at-least-once and unordered but not how to handle it; the answer sheet has 14 open boundaries. The hidden tests are 32 scripted scenarios with explicit duplicates and reorderings, driven through a test-controlled bus, and four property tests that replay 300 seeded random schedules and check that every order settles, stock and money are conserved, and no compensation event is sent twice. A naive implementation without deduplication or memory of cancellations scores 27 of 36.

Each condition ran in two ways. *Solo*: one agent implements all three services, as in T1–T3. *Team*: three agents work in parallel, one per service, each in its own repository with only the shared event types and its own service visible; under Spec Kit and SpecEngine each developer runs the full tool on its own service and asks its own questions, and a script assembles the three files for scoring. Every setting ran with Fable 5.1 and Haiku 5.5 and SpecEngine 2.1.0, for 12 runs and 24 agents; one separate session answered all questions from the sheet.

Table IX gives the results. Data consistency was not where the conditions differed: 11 of the 12 runs passed every duplicate, reordering, and conservation test, including the baseline teams whose developers never saw each other's code. The one consistency failure was Spec Kit's Haiku solo run, whose payment service dropped a stock-reserved event that arrived before the order it belonged to, leaving the order pending and its stock reserved. The baseline teams lost points on decisions made separately in each service: each developer invented its own failure codes. SpecEngine scored full marks in all four settings. With the weak model Spec Kit lost points again, by a smaller margin than on T2, and its losses include the one consistency failure. In SpecEngine's flow review, Haiku recovered boundaries that its own discovery phase had missed: in the solo run, the diagrams showed that only the payment service remembered early cancellations, and the agent

asked about the inventory service; in the team, the payment developer asked the same question about its own service. Neither tool checks the contract between services: the teams stayed consistent because every developer asked the same stakeholder and received the same answers, and nothing in either tool would notice if two teams received different ones. The cost grew with the team: SpecEngine teams used 684k (Fable) and 751k (Haiku) tokens in total, against 176k and 194k without a tool and 579k and 497k with Spec Kit. The Lock held for every SpecEngine project in T4: AUDIT passed on both solo runs and all six team developers' projects, and the tamper script reported LOCK_DRIFT on both solo runs.

## VI. Discussion

### A. What the Results Show

Three findings stand out. First, with strong models, having any SDD tool mattered more than which one: both cleared the baseline by a wide margin on every task, and the baseline's failures were concentrated in exactly the boundaries a requirement leaves open. Second, the two SDD tools differed most under a weak model. Spec Kit's clarify step relies on the model's judgment of which ambiguities matter, and on T2 the weak model judged that none did; SpecEngine's probe list asks the model that question less. Our hypothesis is that procedure, rather than model strength, explains the gap. It is consistent with the observations but rests on single runs, and the weak model still missed boundaries that a strong model asked about (9 of 11 against 11 of 11 on T2) or that only the flow review recovered (T4). In T4 the gap was smaller and had a different cause: Spec Kit's weak solo run asked three questions and its losses include the one consistency failure, and its weak team run differed from SpecEngine's by a single test. In T4, consistency under duplicated and reordered events was handled in 11 of 12 runs once the requirement stated the delivery guarantees; what the tools added was agreement on decisions across services, and that agreement came from a shared stakeholder rather than from any check. Third, the lock detected every specification edit made after completion, at every level. The freeze during implementation (snapshot and verify) and the Driver Interface were exercised only in the case study: no benchmark run met a conflict, and all benchmark runs used direct binding. Two gaps remained open in every condition: example values that are wrong but consistent with the code, and step bodies emptied of assertions. Both point to concrete improvements: storing each rule's preconditions and postconditions as structured data so that a script can derive or check the example tables, and failing a phase when a Then step contains no assertion or when a mutation pass before locking finds surviving specification-relevant mutants.

### B. Remaining Evaluation

The benchmark compares workflows; it does not isolate the contribution of the freeze itself, because in every SpecEngine run the tests went green without a conflict. The case study's incident is one observation. The question that remains is RQ1: with the same model and tasks, do hash-frozen artifacts lead to fewer unrecorded specification edits than a prompt-only prohibition? We propose the following design.

*Conditions and tasks.* Twenty business modules, each with a requirement of three to six sentences containing at least two numeric boundaries and one state transition, written by someone not involved in the design. P1–P5 are shared across conditions and all Decisions are answered in advance, so every condition starts from the same all-failing state and variation comes from P6 alone. Arm F (frozen) uses the full instructions with the scripts; arm P (prompt-only) is identical with the script calls removed; arm U (unconstrained) only asks for the tests to pass. Because a contradiction between an Examples row and its Rule is a sanctioned formalization fix in release 2.0.0, the seeded conflicts must instead be specification defects: in half of the modules, two Rules are made to contradict each other in a way only $S$ can resolve, so that some test cannot pass without a decision. The other half measure the natural rate. Each module–condition pair is repeated five times, for 300 runs.

*Measures.* Unrecorded change rate: the proportion of runs in which any frozen artifact differs between start and end without a corresponding fix record. Recorded fixes are counted separately, as are the two escape hatches: a Then step whose assertion was removed and a driver implementation that does not call product code. The honest-report rate, the false-completion rate (a report of all tests passing while an unrecorded change occurred), and the true-completion rate complete the set. All are computed by an independent script that does not know the condition.

*Analysis.* The primary comparison is F against P on the seeded half, by a Cochran–Mantel–Haenszel test stratified by module. It is cross-checked by a mixed-effects logistic regression fit on all runs, with module as a random effect and arm, seeding, and arm $\times$ seeding as fixed effects. With 50 seeded runs per arm, a reduction from 40% to 10% is detectable at two-sided $\alpha = 0.05$ with power of about 0.94 if runs were independent; but the 50 runs come from 10 modules, and with an intra-module correlation of 0.2 the design effect is 1.8 and power falls to about 0.75. A pilot will therefore estimate the base rate and the intra-module correlation, and the number of modules will be set from them; twenty is a lower bound. Failing to reject the null would not show that the scripts are redundant: an equivalence claim needs a margin fixed in advance and several hundred runs per arm, which is beyond this design.

*Decision quality.* RQ2 asks whether P1's Decisions cover the decision points that senior engineers list independently. Three engineers with at least five years of experience list, for the same requirements, the items an implementation must decide but the requirement leaves unstated; two raters, blind to the source, align items across lists, with their agreement reported as Cohen's $\kappa$. Every source, P1 included, is scored by leave-one-out recall against the union of the items listed by the three engineers other than itself, over the same 20 requirements. The hypothesis is non-inferiority: P1's recall is no more than ten percentage points below the engineers' mean

recall.

### C. Threats to Validity

*Construct validity.* The freeze detects byte changes, which counts meaning-preserving changes such as reformatting and misses tests that are untouched but passed through a wrong generalization. Driver interfaces are recognized by file name; an interface written to a file outside the convention is not frozen. In the benchmark, hidden tests check the fixed interface only, and a boundary counts as asked only if it appears among the agent's questions.

*Internal validity.* The requirements, answer sheets, hidden tests, and evaluation were designed by the tool's author. Stakeholders were simulated from answer sheets, by the same agent in the first runs and by a separate session later; three runs that read ahead were voided. Runs were interrupted by rate limits and resumed, and the T3 hidden tests were corrected once and all T3 runs rescored. In T4 one session answered every developer from one sheet, so teams never received conflicting answers, as real teams can, and up to fifteen agents waited on that session at once, which inflates the times of conditions B and C unevenly. The case study's stakeholder was also a model instance, and its executor was explicitly told to stop, so I4 cannot be generalized from either study.

*External validity.* All runs used one model family and one agent tool, and most cells hold one or two runs, so differences of one or two tests are not evidence of a difference between tools. The weak-model and T4 runs used release 2.1.0 while the T1–T3 strong-model runs used 2.0.0; their phase instructions are identical, but the comparison is not perfectly controlled. Tasks were small (7 to 14 open boundaries, at most three services); Zhao et al. [1] report that hacking grows with code size, and the behavior on large modules is unknown.

*Intrinsic limits of the mechanism.* Tests are examples, not proofs; the freeze prevents changing the question, not passing it with a wrong generalization. The scripts are run by the agent, so the freeze detects rather than prevents a deliberate edit (Section III-E). The formalization-fix route lets the agent edit the Contract on its own judgment (Section III-G). I4 cannot be verified by script. Whether the Contract matches the business outcome lies outside the guarantee, and cross-module outcomes currently belong to no Rule. The shared data model and glossary are frozen byte for byte, so the current design effectively assumes that modules are implemented one at a time.

## VII. Conclusion and Future Work

We framed specification integrity under LLM agents as a conflict among five forces and observed that current specification-driven development tools resolve readability versus executability largely in favor of readability and leave stability versus freedom unresolved. SpecEngine answers both by stepwise refinement into an executable, hash-frozen contract: the freeze is limited to the objects that define correctness (the Contract, the Step Sentences, and the Driver Interface) and leaves implementation free (function bodies, driver implementations, and product code); decision authority is externalized through multiple-choice questions generated by procedure; and state is derived from artifacts. In a case study, the agent's own formalization error collided with the frozen contract, and the agent reported it rather than rewriting the contract. In a public benchmark, both SDD tools far outperformed no tool with strong models and tied with each other, and with a weak model SpecEngine kept full marks while Spec Kit fell to the baseline on one task; on event-driven services built by teams, SpecEngine again scored full marks, but neither tool checked the contracts between services; the lock detected every tampered specification, and fault injection located the two gaps that remain in every condition. The freeze during implementation was exercised only in the case study.

Future work follows from these results. First, the controlled experiment of Section VI-B would isolate the effect of the freeze from that of the workflow. Second, storing each Rule's conditions as structured data would let a script derive or check the example tables, and an assertion check or a mutation pass before locking would close the hollow-step gap. Third, running VERIFY in continuous integration against a snapshot committed before implementation would turn detection into prevention. Fourth, for cross-module and cross-service outcomes, objects and events passed between modules could become first-class entities whose producer postconditions and consumer preconditions cite the same contract identifier, so that a script can detect two teams deciding the same boundary differently. Fifth, moving the right to write Decision answers out of the agent's tool set would make I4 mechanical. Finally, distributing the core with thin adapters for other agent tools, and repeating the benchmark with more runs and other model families, would test tool independence and generality.

## Data Availability

The plugin source code, the case-study project with all of its artifacts and the Lock file, the benchmark (requirements, answer sheets, hidden tests, reference implementations, every run's output and log, and the scoring, tampering, fault-injection, and mutation scripts), and the LaTeX source of this paper are available at https://github.com/yi-luan/specengine-plugin, tag `arxiv-v1`. The release described in Section III is tag `v2.0.0`. The case-study executor's per-round reports are available from the author on request.


## Acknowledgment

Generative AI assistance (Claude, Anthropic) was used in drafting and translating this manuscript; the author reviewed and takes full responsibility for its content. The case study and the benchmark used Claude models as the agents under study, as described in Sections IV and V.


## References


[1] B. Zhao, D. Srikanth, Y. Wu, and Z. Jiang, "SpecBench: Measuring reward hacking in long-horizon coding agents," arXiv:2605.21384, May 2026.

[2] Z. Zhong, A. Raghunathan, and N. Carlini, "ImpossibleBench: Measuring LLMs' propensity of exploiting test cases," arXiv:2510.20270, Oct. 2025.
[3] J. Gabor, J. Lynch, and J. Rosenfeld, "EvilGenie: A reward hacking benchmark," arXiv:2511.21654, Nov. 2025.
[4] B. Böckeler, "Understanding spec-driven-development: Kiro, spec-kit, and Tessl," martinfowler.com, Oct. 15, 2025. [Online]. Available: https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html
[5] D. B. Piskala, "Spec-driven development: From code to contract in the age of AI coding assistants," arXiv:2602.00180, Jan. 2026.
[6] Kiro, "Correctness with property-based tests," Kiro documentation. [Online]. Available: https://kiro.dev/docs/specs/correctness/ (accessed Nov. 1, 2026).
[7] GitHub, "Spec Kit," GitHub repository. [Online]. Available: https://github.com/github/spec-kit (accessed Nov. 1, 2026).
[8] N. Wirth, "Program development by stepwise refinement," *Commun. ACM*, vol. 14, no. 4, pp. 221–227, Apr. 1971.
[9] D. North, "Introducing BDD," *Better Software*, Mar. 2006. [Online]. Available: https://dannorth.net/introducing-bdd/
[10] J. F. Smart and J. Molak, *BDD in Action*, 2nd ed. Shelter Island, NY, USA: Manning, 2023.
[11] Cucumber, "Gherkin reference." [Online]. Available: https://cucumber.io/docs/gherkin/reference/ (accessed Oct. 20, 2026).
[12] M. Wynne, "Introducing Example Mapping," Cucumber Blog, Dec. 8, 2015. [Online]. Available: https://cucumber.io/blog/bdd/example-mapping-introduction/
[13] G. Adzic, *Specification by Example: How Successful Teams Deliver the Right Software*. Shelter Island, NY, USA: Manning, 2011.
[14] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA, USA: Addison-Wesley, 2003.
[15] C. A. R. Hoare, "An axiomatic basis for computer programming," *Commun. ACM*, vol. 12, no. 10, pp. 576–580, 583, Oct. 1969.
[16] B. Meyer, "Applying 'design by contract'," *Computer*, vol. 25, no. 10, pp. 40–51, Oct. 1992.
[17] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation*. Boston, MA, USA: Addison-Wesley, 2010.
[18] D. Farley, *Modern Software Engineering*. Boston, MA, USA: Addison-Wesley, 2021.
[19] K. Beck, *Test-Driven Development: By Example*. Boston, MA, USA: Addison-Wesley, 2002.
[20] S. Freeman and N. Pryce, *Growing Object-Oriented Software, Guided by Tests*. Boston, MA, USA: Addison-Wesley, 2009.
[21] M. Jackson, *Problem Frames: Analysing and Structuring Software Development Problems*. Boston, MA, USA: Addison-Wesley, 2001.
[22] C. Alexander, *The Timeless Way of Building*. New York, NY, USA: Oxford Univ. Press, 1979.
[23] S. Karpurapu, S. Myneni, U. Nettur, L. S. Gajja, D. Burke, T. Stiehm, and J. Payne, "Comprehensive evaluation and insights into the use of large language models in the automation of behavior-driven development acceptance test formulation," arXiv:2403.14965, Mar. 2024.
[24] A. Rathnayake, M. Shahin, and G. Abaei, "Behaviour driven development scenario generation with large language models," arXiv:2603.04729, Mar. 2026.
[25] S. Siddeeq, M. Abbasi, J. Rasku, Z. Zhang, F. Christophe, T. Mikkonen, and P. Abrahamsson, "Epic-organized vs. requirement-aligned Gherkin: An empirical evaluation of LLM-based acceptance criteria generation," arXiv:2607.01980, Jul. 2026.
[26] S. Hassani, M. Sabetzadeh, and D. Amyot, "From law to Gherkin: A human-centred quasi-experiment on the quality of LLM-generated behavioural specifications from food-safety regulations," arXiv:2508.20744, Aug. 2025.
[27] S. K. Lahiri, S. Fakhoury, A. Naik, G. Sakkas, S. Chakraborty, M. Musuvathi, P. Choudhury, C. von Veh, J. P. Inala, C. Wang, and J. Gao, "Interactive code generation via test-driven user-intent formalization," arXiv:2208.05950, Aug. 2022.
[28] S. Fakhoury, A. Naik, G. Sakkas, S. Chakraborty, and S. K. Lahiri, "LLM-based test-driven interactive code generation: User study and empirical evaluation," arXiv:2404.10100, Apr. 2024.
[29] V. Wahi, "LLM-as-a-judge is not an oracle: Why self-improving agents need deterministic guardrails," arXiv:2609.02246, Sep. 2026.
[30] P. Runeson and M. Höst, "Guidelines for conducting and reporting case study research in software engineering," *Empirical Softw. Eng.*, vol. 14, no. 2, pp. 131–164, 2009.